



Music Plagiarism Detector

Maria Quinteiro & Inês Batista
124996/124877

1º entrega 24/05

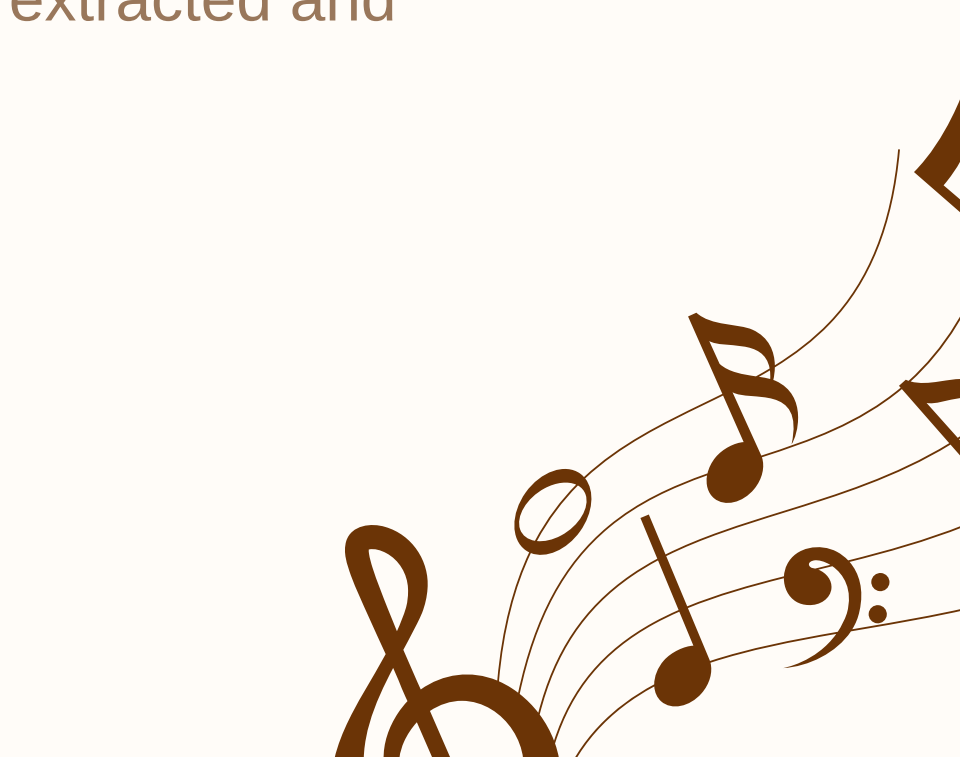




Introduction

This project aims to develop a tool to detect possible cases of musical plagiarism through melody comparison. The main approach uses k-shingles to represent small melody fragments and compare those fragments between different songs.

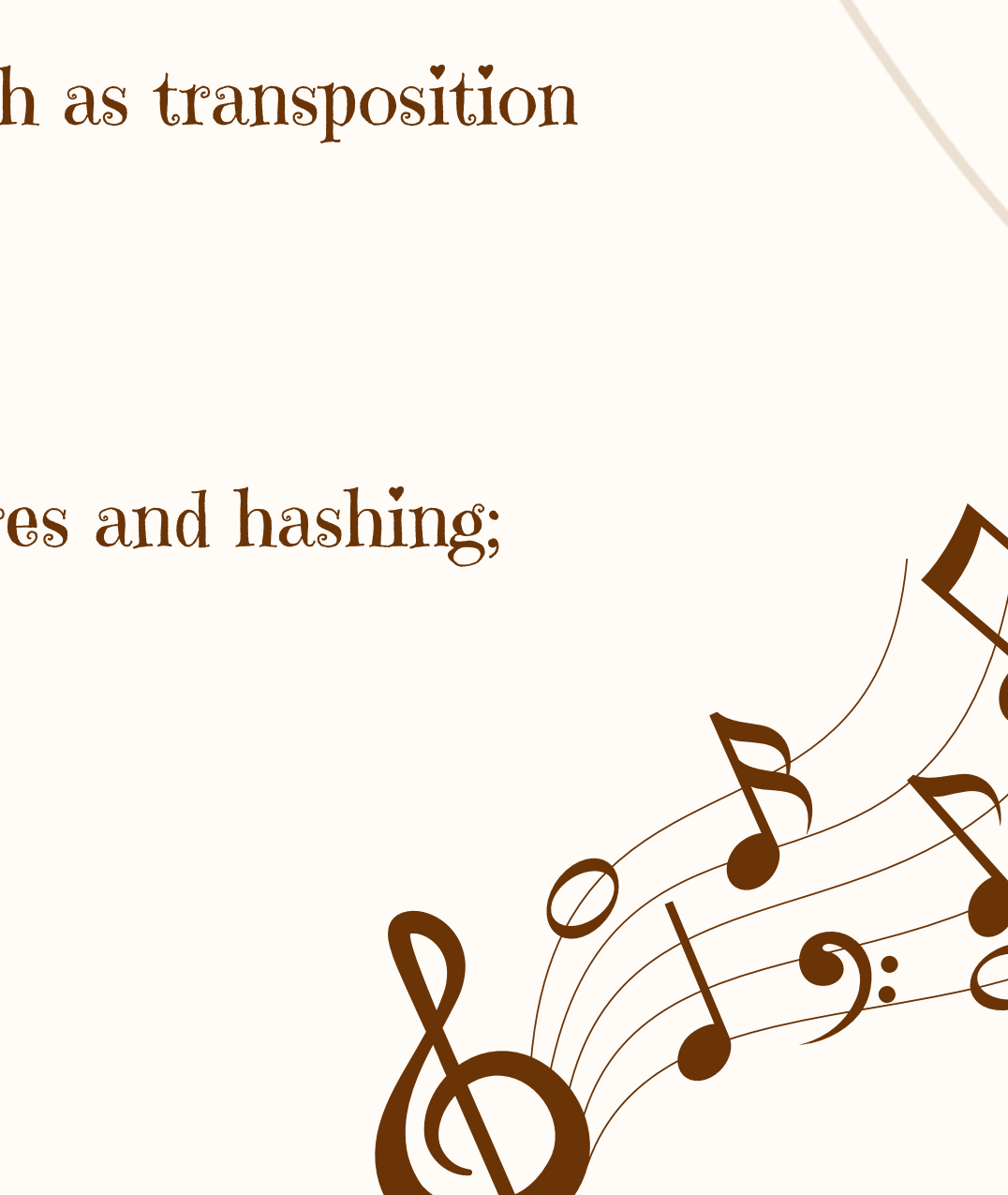
The system works on a simplified representation of melody: sequences of notes and their respective durations. From this representation, pitch and rhythm features are extracted and then converted into comparable shingle sets.





Objectives



- Represent melodies in a simple and comparable way;
 - Normalize melodies to reduce irrelevant differences, such as transposition or small rhythmic variations;
 - Generate shingles based on sequences of 8 elements;
 - Compare melodies efficiently using probabilistic structures and hashing;
- 
- 



Implemented Modules



- **Preprocessing**

The preprocessing.m module prepares melodies for comparison by cleaning and normalizing notes, durations, and rhythm patterns, making melodies comparable even when played in different keys.

- **Shingle Generation**

The shingles.m module generates melodic fragments of 8 notes using pitch and rhythm information, converting them into hashed numeric values for efficient melody comparison while reducing sensitivity to small rhythmic variations.



Implemented Modules



- **Bloom Filter**

The `bloom_filter.m` module uses a Bloom Filter to quickly check whether melodic shingles may exist in a reference melody, providing fast probabilistic filtering with possible false positives but no false negatives.

- **MinHash and LSH**

The `minhash_lsh.m` module finds similar melodies efficiently by using MinHash signatures and Locality Sensitive Hashing to detect candidate pairs, which are then validated using Jaccard similarity.





Implemented Modules



- **Naïve Bayes**

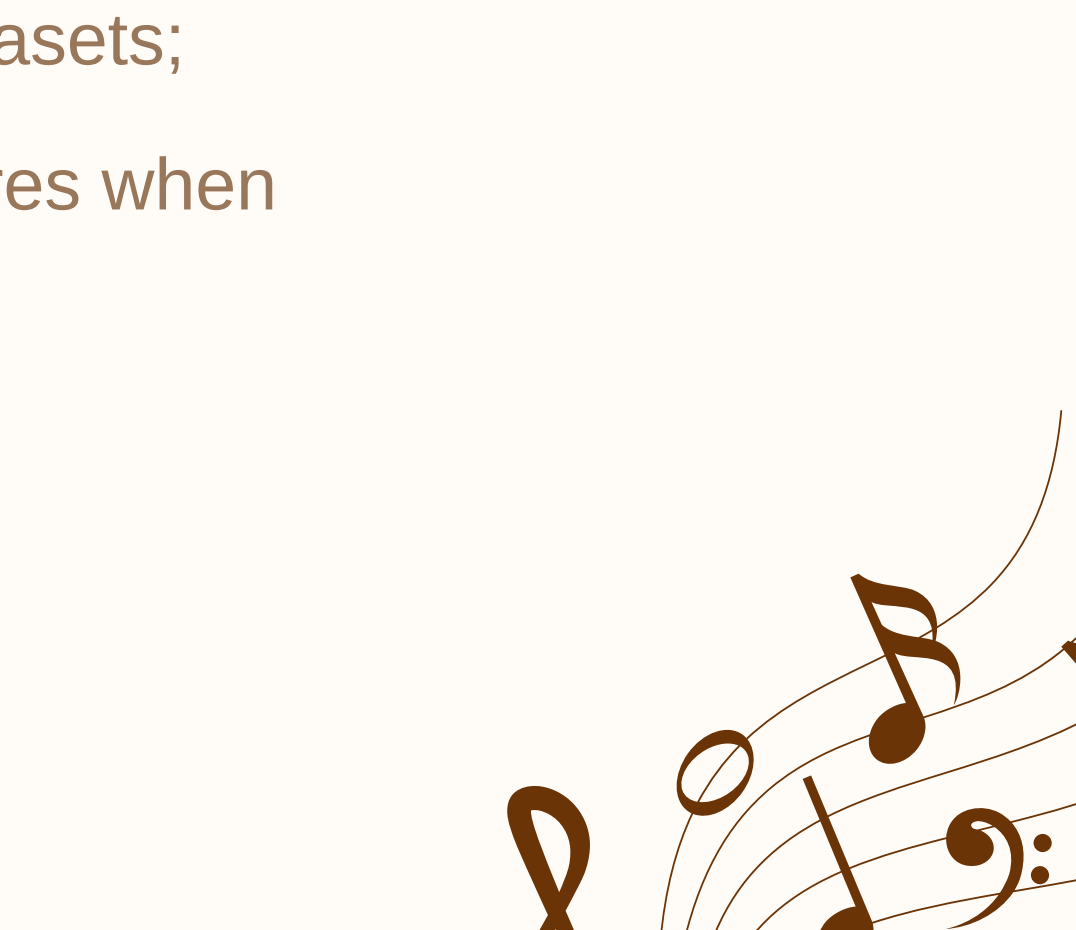
The `naive_bayes.m` module implements a Multinomial Naive Bayes classifier that uses extracted melody features or shingle hashes to classify melodies based on labeled training examples.





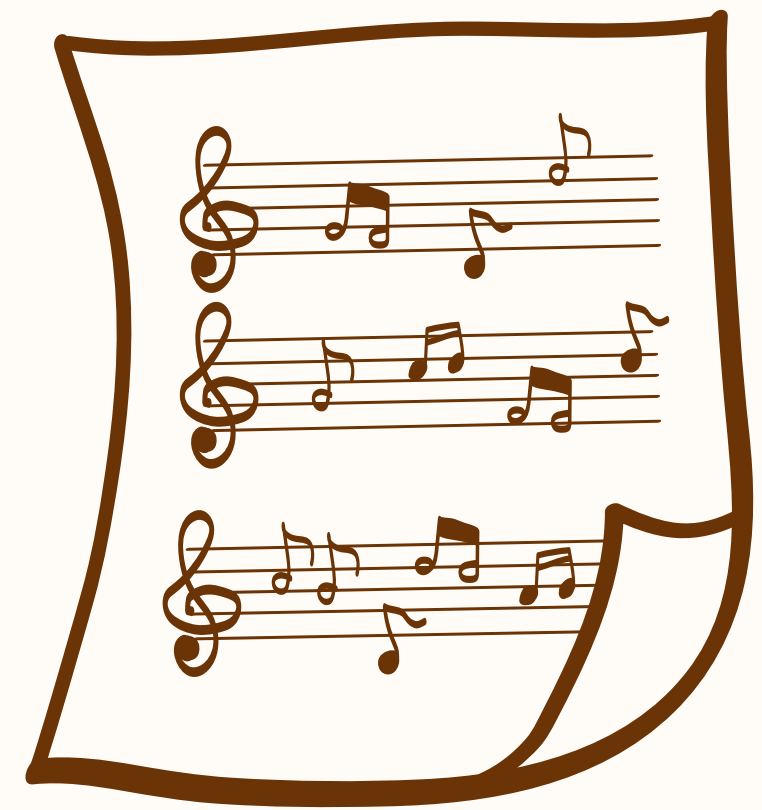
Planned Pipeline



1. Load or represent a melody as a sequence of notes and durations;
 2. Apply preprocessing to obtain intervals and normalized rhythm;
 3. Generate shingles of size 8;
 4. Compare shingles using a Bloom Filter for fast filtering;
 5. Use MinHash + LSH to find similar candidates in larger datasets;
 6. Use Naive Bayes to classify melodies from extracted features when labeled data is available;
 7. Validate candidates using Jaccard similarity;
 8. Produce a final decision or possible plagiarism score.
- 
- 

Next Steps

1. Implement detection.m, integrating preprocessing, shingles, Bloom Filter, and MinHash/LSH.
2. Implement main.m to execute the complete pipeline
3. Validate the system with simple artificial melodies before using real data.
4. Add support for loading musical files.



Conclusion

The project already includes the main algorithmic foundation for comparing and classifying melodies using techniques such as shingles, Bloom Filters, MinHash/LSH, Jaccard similarity, and Naive Bayes classification.

The current implementation supports melody representation, normalization, efficient similarity detection, and feature-based classification of melodic fragments. However, the modules still need to be fully integrated in order to create a complete musical plagiarism detection system.

